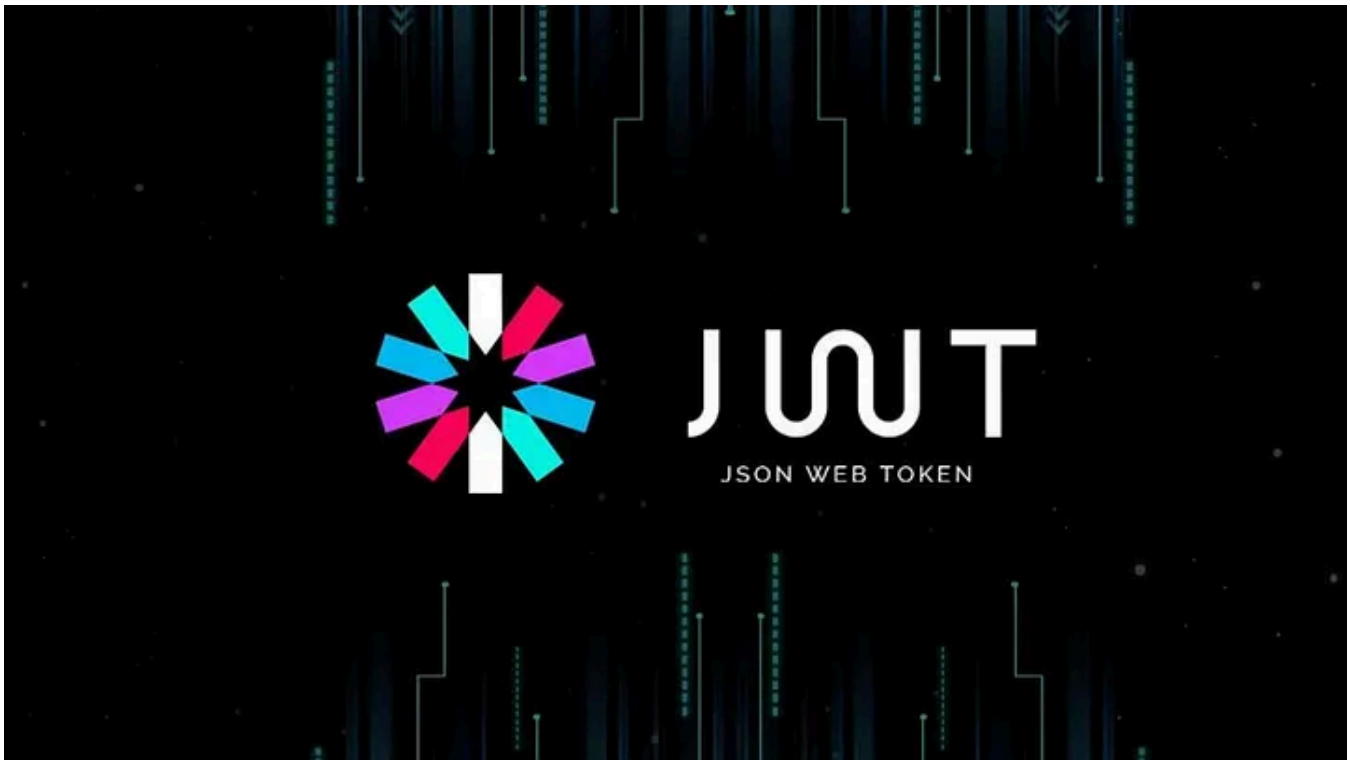
Expressjs + Jwt Authentication + Jwt + Typescript + Prisma +

Step-by-Step Guide to Secure JWT Authentication with Refresh Tokens in Express.js, TypeScript, and Prisma

22 min read · Mar 18, 2025

gigi shalamberidze Follow

Listen Share ••• More



Learn how to implement secure authentication and authorization in an Express.js API using JWT, TypeScript, and Prisma. This guide walks you through setting up access & refresh tokens, securing endpoints, and structuring a scalable project

with controllers, middlewares, and validations. Perfect for building authentication in real-world apps!

You'll learn how to:

1. Securely generate, store, and validate **access tokens** and **refresh tokens**
2. Implement **middleware-based authentication** to protect API routes
3. Handle **user login, registration, and logout** with proper token revocation
4. **Structure your Express.js project** for scalability using controllers, middlewares, and validations

🛠️ Getting Started: Installation & Setup

Before diving into the code, let's set up the project and install the necessary dependencies.

Install Node.js & npm (If Not Installed)

First, make sure you have **Node.js** and **npm** installed on your system.

🔹 **Check if Node.js & npm are installed** by running the following command in your terminal:

If these commands return a version number (e.g., v18.16.0 for Node.js), you're good to go! 🎯

❌ **If not installed**, download and install the latest **LTS version** of Node.js from:

🔗 https://nodejs.org/

1️⃣ Initialize a New Express.js Project

Once Node.js and npm are installed, follow these steps:

📌 Create a new project folder & navigate into it:

```
mkdir express-auth-prisma  # creates express-auth-prisma folder
cd express-auth-prisma     # navigate into express-auth-prisma folder
```

📌 **Initialize the project with npm**

```
npm init -y
```

📌 **Install Required Dependencies**

Install the core dependencies for your project:

```
npm install cookie-parser cors dotenv express jsonwebtoken bcryptjs zod @prisma
```

What Each Dependency Does:

1. **cookie-parser:** Parses cookies attached to incoming HTTP requests. Why It's Needed: Essential for handling authentication tokens stored in cookies (e.g., JWT tokens).
2. **cors:** Enables Cross-Origin Resource Sharing (CORS). Why It's Needed: Allows your API to handle requests from different domains (e.g., a frontend app running on http://localhost:3000).
3. **dotenv:** Loads environment variables from a .env file into process.env. Why It's Needed: Keeps sensitive information (e.g., database URLs, secret keys) out of your codebase.
4. **express:** A minimal and flexible Node.js web application framework. Why It's Needed: The backbone of your API, handling routing, middleware, and HTTP requests/responses.
5. **jsonwebtoken:** Generates and verifies JSON Web Tokens (JWT). Why It's Needed: Used for creating secure access and refresh tokens for authentication

6. **bcryptjs:** Hashes passwords securely. Why It's Needed: Protects user passwords by storing them as hashed values in the database.
7. **zod:** A TypeScript-first schema validation library. Why It's Needed: Validates user input (e.g., login/registration data) to ensure it meets required criteria.
8. **@prisma/client:** An auto-generated database client for Prisma. Why It's Needed: Provides a type-safe way to interact with your database (e.g., querying, inserting, updating data).

📌 Install Development Dependencies

Add the necessary dev dependencies for TypeScript and Prisma:

```
npm install -D @types/cookie-parser @types/cors @types/express @types/node @typ
```

What Each Dependency Does:

1. **@types/ Packages (e.g., @types/cookie-parser, @types/cors, etc.):** TypeScript type definitions for Node.js and third-party libraries. Why It's Needed: Provides TypeScript with type information for JavaScript libraries, enabling better type checking and autocompletion.
2. **nodemon:** Automatically restarts the server when file changes are detected. Why It's Needed: Speeds up development by eliminating the need to manually restart the server after code changes.
3. **prisma:** A modern database toolkit for TypeScript and Node.js. Why It's Needed: Manages database migrations, generates a type-safe client, and simplifies database interactions.
4. **ts-node:** Enables running TypeScript files directly without pre-compiling to JavaScript. Why It's Needed: Simplifies development by allowing you to execute .ts files on the fly.
5. **tsconfig-paths:** Resolves custom path aliases defined in tsconfig.json. Why It's Needed: Allows you to use cleaner import paths (e.g., @utils/response instead of ../../utils/response)

6. **tsc-alias:** Replaces path aliases with the correct paths in the built JavaScript files. **Why It's Needed:** Ensures that the path aliases you use during development are correctly resolved in the compiled JavaScript output.

📌 Initialize Typescript

Generate a tsconfig.json file to configure TypeScript:

```
npx tsc --init
```

📌 Set Up Prisma

Set up Prisma in your project:

```
npx prisma init
```

This creates a .env file for environment variables and a prisma folder with a schema.prisma file for database schema definitions.

📌 Configure package.json Scripts

Update the scripts section in package.json to include Nodemon for automatic server restarts:

```
"scripts": {
  "dev": "nodemon ./src/index.ts",
  "build": "tsc && tsc-alias",
  "start": "node ./dist/index.js"
},
```

📌 Set Up Path Aliases

Path aliases make your imports cleaner and more readable by allowing you to use custom names instead of long relative paths (../../utils/someFile.ts).

Step 1: Open tsconfig.json

Find the compilerOptions section and add the following:

```
"outDir": "./dist",
"rootDir": "./src",
"baseUrl": "./src",
"paths": {
  "@config/*": ["config/*"],
  "@middlewares/*": ["middlewares/*"],
  "@controllers/*": ["controllers/*"],
  "@routes/*": ["routes/*"],
  "@utils/*": ["utils/*"]
},
```

This allows you to use cleaner imports like @utils/response instead of long relative paths.

Explanation:

1. outDir: Specifies that compiled TypeScript files should be outputted to the dist folder.
2. rootDir: Indicates that all TypeScript source files are inside the src directory.
3. baseUrl: Sets src as the base directory for module resolution.
4. paths: Defines alias shortcuts to avoid long relative imports (../../utils/response.utils.ts → @utils/response.utils.ts).

📌 Define Your Prisma Schema

In prisma/schema.prisma, define the User model:

```
// This is your Prisma schema file,
// learn more about it in the docs: https://pris.ly/d/prisma-schema

// Looking for ways to speed up your queries, or scale easily with your serverl
// Try Prisma Accelerate: https://pris.ly/cli/accelerate-init

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql" // you can change this to use other databases like Postgre
  url      = env("DATABASE_URL") // Your database connection URL (stored in .er
}
```

```
model User {
  id           Int      @id @default(autoincrement())
  username     String   @unique
  email        String   @unique
  password     String
  refreshToken String?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}
```

This schema sets up a basic user model with fields for authentication and token management.

!!! In datasource db object you can replace database provider based on your database in my case i will use mysql

2️⃣ Folder Structure & Explanation

A well-organized folder structure is key to maintaining a clean and scalable codebase. Here's the recommended structure:

```
📦 express-auth-prisma
 ├ 📜 node_modules/
 ├ 📜 prisma
 ├ 📂 src
 │ ├ 📂 config
 │ │ ├ 📜 app.config.ts
 │ │ ├ 📜 auth.config.ts
 │ ├ 📂 controllers
 │ │ ├ 📜 auth.controller.ts
 │ │ ├ 📜 user.controller.ts
 │ ├ 📂 middlewares
 │ │ ├ 📜 auth.middleware.ts
 │ │ ├ 📜 validation.middleware.ts
 │ ├ 📂 routes
 │ │ ├ 📜 auth.routes.ts
 │ │ ├ 📜 router.ts
 │ │ ├ 📜 user.routes.ts
 │ ├ 📂 utils
 │ │ ├ 📜 response.utils.ts
 │ ├ 📂 validations
 │ │ ├ 📜 auth.schema.ts
 │ ├ 📜 app.ts
 │ ├ 📜 db.ts
 │ ├ 📜 index.ts
 ├ 📜 .env
```

```
├ 📜 package-lock.json
├ 📜 package.json
├ 📜 tsconfig.json
```

Project Structure & File Explanations

📂 **Root Directory**

1. **node_modules/:** Contains all installed npm packages and dependencies. prisma/: Stores Prisma-related files, including the schema.prisma file for defining your database schema.

2. **src/:** The main source folder containing all backend logic and application code. .env: Stores environment variables like database URLs, API keys, and secret tokens. package.json & package-lock.json: Manage project dependencies and scripts. tsconfig.json: Configures TypeScript settings for the project.

📂 **src/ (Main Source Folder)**

This folder contains all the backend logic and Express app configuration.

1. **config/:** Stores configuration files for the application.

- **app.config.ts:** Manages application settings like host and port
- **auth.config.ts:** Handles JWT secrets and token expiration settings.

2. **controllers/:** Contains functions that handle route requests and business logic.

- **auth.controller.ts:** Manages authentication-related operations (login, registration, logout, token refresh).
- **user.controller.ts:** Handles user-related operations (e.g., fetching user info).

3. **middlewares/:** Holds Express middleware functions for processing requests.

- **auth.middleware.ts:** Authenticates users using JWT tokens.
- **validation.middleware.ts:** Validates request data using Zod schemas.

4. **routes/:** Defines route handlers for modular organization.

- **auth.routes.ts:** Routes for authentication endpoints (e.g., /login, /register).
- **user.routes.ts:** Routes for user-related endpoints (e.g., /user/info).

- **router.ts:** Base router class for defining and registering routes.

5. utils/: Stores utility and helper functions for reusable logic.

- **response.utils.ts:** Provides utility methods for sending consistent API responses.

6. validations/: Manages Zod schema validation for request body data.

- auth.schema.ts: Defines validation schemas for authentication-related requests (e.g., login, registration).

7. app.ts: Configures the Express application, initializes middlewares, and registers routes.

8. db.ts: Configures the Prisma client for database interactions. Key Features: Ensures a singleton instance of the Prisma client for efficient database connections. Prevents unnecessary re-initialization in development environments.

9. index.ts: The entry point of the application. Bootstraps and starts the server.

3️⃣ Code Part: Implementing Authentication and Configuration

Now that we've set up the project and installed the necessary dependencies, let's dive into the code! Below, you'll find the implementation details for configuration files, controllers, and other essential components.

⚠️ ⚠️ ⚠️ Note While you can copy and paste the code provided in this section, some imports may not work immediately. This could be due to:

1. **Unresolved TypeScript Path Aliases:** Imports like import Send from "@utils/response.utils" won't work until we configure TypeScript path aliases in the next section.

2. **Missing Files:** Some code references files (e.g., controllers, utilities, or validations) that haven't been created or copied yet.

Don't worry — we'll address these issues step by step in the upcoming sections. For now, focus on understanding the logic and flow of the code.

🔧 Configuration Files

📁 src/.env

The .env file stores environment-specific configuration values, such as database credentials and authentication secrets. These values are loaded into the application

at runtime, keeping sensitive information secure and out of your codebase.

```
# Application Settings
APP_HOST=localhost
APP_PORT=8000

# Database Configuration
DATABASE_URL="mysql://db-username:db-password@localhost:3306/db-name" # db conn

# Authentication Settings
AUTH_SECRET="your_jwt_access_token_secret"  # Secret key for signing JWT access
AUTH_SECRET_EXPIRES_IN="15m"                # Access token expiration time (15
AUTH_REFRESH_SECRET="your_jwt_refresh_token_secret"  # Secret key for signing J
AUTH_REFRESH_SECRET_EXPIRES_IN="24h"        # Refresh token expiration time (24
```

🚀 Run Prisma Migrations

After defining your **User model** in prisma/schema.prisma, you need to run **Prisma Migrate** to create the necessary tables in your database.

Step1: Run Prisma migrate

This command will:

- Generate a migration file in the `prisma/migrations` folder.
- Apply the migration to your database, creating the `User` table.

```
npx prisma migrate dev --name add-user-schema
```

Step 2: Verify the Migration

1. Check the `prisma/migrations` folder to ensure the migration file was created.
2. Open your database (e.g., MySQL, PostgreSQL) and verify that the `User` table was created with the correct columns.

Step 3: Regenerate the Prisma Client

After running the migration, regenerate the Prisma Client to ensure it reflects the latest schema changes:

```
npx prisma generate
```

📁 **src/config/app.config.ts**

```
const appConfig = {
    host: process.env.APP_HOST as string,
    port: parseInt(process.env.APP_PORT as string),
}

export default appConfig;
```

📁 **src/config/auth.config.ts**

```
const authConfig = {
    // Secret key used for signing JWT access tokens
    secret: process.env.AUTH_SECRET as string,

    // Expiration time for the JWT access token (e.g., "15m" for 15 minutes)
    secret_expires_in: process.env.AUTH_SECRET_EXPIRES_IN as string,

    // Secret key used for signing JWT refresh tokens
    refresh_secret: process.env.AUTH_REFRESH_SECRET as string,

    // Expiration time for the JWT refresh token (e.g., "24h" for 24 hours)
    refresh_secret_expires_in: process.env.AUTH_REFRESH_SECRET_EXPIRES_IN as st
}

export default authConfig;
```

Why These Configurations Matter?

1. **Security:** Sensitive data like database URLs and JWT secrets are stored securely in .env
2. **Flexibility**: Environment variables make it easy to switch configurations between development, testing, and production environments.
3. **Scalability:** Centralized configuration files like app.config.ts and auth.config.ts simplify updates and maintenance.

🛠️ Controllers: Handling Authentication and User Operations.

Controllers are the backbone of your application, responsible for handling incoming requests, processing business logic, and returning responses. Below, we'll explore the auth.controller and user.controller in detail.

📁 src/controllers/auth.controller.ts

The AuthController manages all authentication-related operations, including login, registration, logout, and token refresh.

```
import Send from "@utils/response.utils";
import { prisma } from "db";
import { Request, Response } from "express";
import authSchema from "validations/auth.schema";
import bcrypt from "bcryptjs";
import { z } from "zod";
import jwt from "jsonwebtoken";
import authConfig from "@config/auth.config";

class AuthController {
    static login = async (req: Request, res: Response) => {
        // Destructure the request body into the expected fields
        const { email, password } = req.body as z.infer<typeof authSchema.logir

        try {
            // 1. Check if the email already exists in the database
            const user = await prisma.user.findUnique({
                where: { email }
            });
            // If user does not exist, return an error
            if (!user) {
                return Send.error(res, null, "Invalid credentials");
            }

            // 2. Compare the provided password with the hashed password storec
            const isPasswordValid = await bcrypt.compare(password, user.passwor
            if (!isPasswordValid) {
                return Send.error(res, null, "Invalid credentials.");
            }

            // 3. Generate an access token (JWT) with a short expiration time (
            const accessToken = jwt.sign(
                { userId: user.id },
                authConfig.secret,  // Use the secret from the authConfig for s
                { expiresIn: authConfig.secret_expires_in as any }  // Use the
            );

            // 4. Generate a refresh token with a longer expiration time (e.g.,
            const refreshToken = jwt.sign(
```

```
                { userId: user.id, },
                authConfig.refresh_secret,  // Use the separate secret for sign
                { expiresIn: authConfig.refresh_secret_expires_in as any }  //
            );

            // 5. Store the refresh token in the database (optional)
            await prisma.user.update({
                where: { email },
                data: { refreshToken }
            });

            // 6. Set the access token and refresh token in HttpOnly cookies
            // This ensures that the tokens are not accessible via JavaScript a
            // The access token expires quickly and is used for authenticating
            // The refresh token is stored to allow renewing the access token w

            res.cookie("accessToken", accessToken, {
                httpOnly: true,   // Ensure the cookie cannot be accessed via J
                secure: process.env.NODE_ENV === "production",  // Set to true
                maxAge: 15 * 60 * 1000,  // 15 minutes in mileseconds
                sameSite: "strict"  // Ensures the cookie is sent only with req
            });
            res.cookie("refreshToken", refreshToken, {
                httpOnly: true,
                secure: process.env.NODE_ENV === "production",
                maxAge: 24 * 60 * 60 * 1000,  // 24 hours is mileseconds
                sameSite: "strict"
            });

            // 7. Return a successful response with the user's basic informatic
            return Send.success(res, {
                id: user.id,
                username: user.username,
                email: user.email
            });

        } catch (error) {
            // If any error occurs, return a generic error response
            console.error("Login Failed:", error); // Log the error for debuggi
            return Send.error(res, null, "Login failed.");
        }

    }
    static register = async (req: Request, res: Response) => {
        // Destructure the request body into the expected fields
        const { username, email, password, password_confirmation } = req.body a

        try {
            // 1. Check if the email already exists in the database
            const existingUser = await prisma.user.findUnique({
                where: { email }
            });
            // If a user with the same email exists, return an error response
```

```
            if (existingUser) {
                return Send.error(res, null, "Email is already in use.");
            }

            // 2. Hash the password using bcrypt to ensure security before stor
            const hashedPassword = await bcrypt.hash(password, 10);

            // 3. Create a new user in the database with hashed password
            const newUser = await prisma.user.create({
                data: {
                    username,
                    email,
                    password: hashedPassword
                }
            });

            // 4. Return a success response with the new user data (excluding p
            return Send.success(res, {
                id: newUser.id,
                username: newUser.username,
                email: newUser.email
            }, "User successfully registered.");

        } catch (error) {
            // Handle any unexpected errors (e.g., DB errors, network issues)
            console.error("Registration failed:", error); // Log the error for
            return Send.error(res, null, "Registration failed.");
        }

    }
    static logout = async (req: Request, res: Response) => {
        try {
            // 1. We will ensure the user is authenticated before running this
            //    The authentication check will be done in the middleware (see
            //    The middleware will check the presence of a valid access toke

            // 2. Remove the refresh token from the database (optional, if usir
            const userId = (req as any).user?.userId;  // Assumed that user dat
            if (userId) {
                await prisma.user.update({
                    where: { id: userId },
                    data: { refreshToken: null }  // Clear the refresh token fr
                });
            }

            // 3. Remove the access and refresh token cookies
            // We clear both cookies here (accessToken and refreshToken)
            res.clearCookie("accessToken");
            res.clearCookie("refreshToken");

            // 4. Send success response after logout
            return Send.success(res, null, "Logged out successfully.");
```

```
        } catch (error) {
            // 5. If an error occurs, return an error response
            console.error("Logout failed:", error); // Log the error for debugg
            return Send.error(res, null, "Logout failed.");
        }
    }
    static refreshToken = async (req: Request, res: Response) => {
        try {
            const userId = (req as any).userId;  // Get userId from the refresh
            const refreshToken = req.cookies.refreshToken;  // Get the refresh

            // Check if the refresh token has been revoked
            const user = await prisma.user.findUnique({
                where: { id: userId }
            });

            if (!user || !user.refreshToken) {
                return Send.unauthorized(res, "Refresh token not found");
            }

            // Check if the refresh token in the database matches the one from
            if (user.refreshToken !== refreshToken) {
                return Send.unauthorized(res, { message: "Invalid refresh token
            }

            // Generate a new access token
            const newAccessToken = jwt.sign(
                { userId: user.id },
                authConfig.secret,
                { expiresIn: authConfig.secret_expires_in as any }
            );

            // Send the new access token in the response
            res.cookie("accessToken", newAccessToken, {
                httpOnly: true,
                secure: process.env.NODE_ENV === "production",
                maxAge: 15 * 60 * 1000,  // 15 minutes
                sameSite: "strict"
            });

            return Send.success(res, { message: "Access token refreshed success

        } catch (error) {
            console.error("Refresh Token failed:", error);
            return Send.error(res, null, "Failed to refresh token");
        }
    }
}

export default AuthController;
```

📁 **src/controllers/user.controller.ts**

The UserController handles user-related operations, such as fetching user information.

```
import Send from "@utils/response.utils";
import { prisma } from "db";
import { Request, Response } from "express";
import { send } from "process";

class UserController {
    /**
     * Get the user information based on the authenticated user.
     * The userId is passed from the AuthMiddleware.
     */
    static getUser = async (req: Request, res: Response) => {
        try {
            const userId = (req as any).userId; // Extract userId from the auth

            // Fetch the user data from the database (Prisma in this case)
            const user = await prisma.user.findUnique({
                where: { id: userId },
                select: {
                    id: true,
                    username: true,
                    email: true,
                    createdAt: true,
                    updatedAt: true,
                    // Add other fields you want to return
                }
            });

            // If the user is not found, return a 404 error
            if (!user) {
                return Send.notFound(res, {}, "User not found");
            }

            // Return the user data in the response
            return Send.success(res, { user });
        } catch (error) {
            console.error("Error fetching user info:", error);
            return Send.error(res, {}, "Internal server error");
        }
    };
}

export default UserController;
```

🛠️ Middleware: Validation and Authentication

Middleware functions sit between the incoming request and the final route handler, allowing you to process requests, validate data, and enforce security checks. Below, we'll explore the validation.middleware and auth.middleware.

📁 src/middlewares/validation.middleware.ts

This middleware validates incoming request data using Zod schemas. If the data doesn't match the schema, it returns a formatted error response.

Key Features:

1. validateBody(schema): Validates the request body against a Zod schema.
2. Formats validation errors into a clean, user-friendly structure (e.g., { email: ["Invalid email"], password: ["Password too short"] }).
3. Sends a 422 Unprocessable Entity response if validation fails.

```
import Send from "@utils/response.utils";
import { NextFunction, Request, Response } from "express";
import { ZodError, ZodSchema } from "zod";

class ValidationMiddleware {
    static validateBody(schema: ZodSchema) {
        return (req: Request, res: Response, next: NextFunction) => {
            try {
                schema.parse(req.body);
                next();
            } catch (error) {
                if (error instanceof ZodError) {
                    // Format errors like { email: ['error1', 'error2'], passwo
                    const formattedErrors: Record<string, string[]> = {};

                    error.errors.forEach((err) => {
                        const field = err.path.join("."); // Get the field name
                        if (!formattedErrors[field]) {
                            formattedErrors[field] = [];
```

```
                        }
                        formattedErrors[field].push(err.message); // Add valida
                    });

                    return Send.validationErrors(res, formattedErrors);
                }

                // If it's another type of error, send a generic error response
                return Send.error(res, "Invalid request data");
            }
        };
    }
}

export default ValidationMiddleware;
```

📁 **src/controllers/auth.middleware.ts**

This middleware handles user authentication and token validation.

Key Methods:

1. authenticateUser Purpose: Verifies the access token stored in an HTTP-only cookie. Steps: Extracts the access token from the cookie. Verifies the token using the JWT secret. Attaches the user's ID to the request object for use in downstream controllers. Returns a 401 Unauthorized error if the token is missing, invalid, or expired.
2. refreshTokenValidation Purpose: Validates the refresh token stored in an HTTP-only cookie. Steps: Extracts the refresh token from the cookie. Verifies the token using the refresh token secret. Attaches the user's ID to the request object. Returns a 401 Unauthorized error if the token is missing, invalid, or expired.
3. Why It's Useful: Protects routes by ensuring only authenticated users can access them. Handles token validation securely using HTTP-only cookies.

```
import authConfig from "@config/auth.config";
import Send from "@utils/response.utils";
import { NextFunction, Request, Response } from "express";
import jwt from "jsonwebtoken";

export interface DecodedToken {
    userId: number;
}
```

```
class AuthMiddleware {
    /**
     * Middleware to authenticate the user based on the access token stored in
     * This middleware will verify the access token and attach the user informa
     */
    static authenticateUser = (req: Request, res: Response, next: NextFunction)
        // 1. Extract the access token from the HttpOnly cookie
        const token = req.cookies.accessToken;

        // If there's no access token, return an error
        if (!token) {
            return Send.unauthorized(res, null);  // Sends 401 Unauthorized if
        }

        try {
            // 2. Verify the token using the secret from the auth config
            const decodedToken = jwt.verify(token, authConfig.secret) as Decode

            // If the token is valid, attach user information to the request ob
            (req as any).userId = decodedToken.userId; // Attach userId to the

            // Proceed to the next middleware or route handler
            next();
        } catch (error) {
            // If the token verification fails (invalid or expired token), retu
            console.error("Authentication failed:", error);  // Log error for d
            return Send.unauthorized(res, null);  // Sends 401 Unauthorized if
        }
    };

    static refreshTokenValidation = (req: Request, res: Response, next: NextFur
        // 1. Extract the refresh token from the HttpOnly cookie
        const refreshToken = req.cookies.refreshToken;

        // If there's no refresh token, return an error
        if (!refreshToken) {
            return Send.unauthorized(res, { message: "No refresh token provided
        }

        try {
            // 2. Verify the refresh token using the secret from the auth confi
            const decodedToken = jwt.verify(refreshToken, authConfig.refresh_se

            // If the token is valid, attach user information to the request ob
            (req as any).userId = decodedToken.userId;

            // Proceed to the next middleware or route handler
            next();
        } catch (error) {
            // Handle token verification errors (invalid or expired token)
            console.error("Refresh Token authentication failed:", error);
```

```
            // Return a 401 Unauthorized with a more specific message
            return Send.unauthorized(res, { message: "Invalid or expired refres
        }
    };
}

export default AuthMiddleware;
```

🛠️ Routes: Organizing API Endpoints

Routes define the entry points for your API, mapping HTTP methods (e.g., GET, POST) to specific controller functions. This section explains how to structure and implement routes using a modular and scalable approach.

📁 src/routes/router.ts

The BaseRouter class is an abstract base class that simplifies route registration. It ensures consistency and reusability across your application.

Key Features:

1. RouteMethod Type: Defines the allowed HTTP methods for routes (get, post, put, delete, patch).
2. RouteConfig Interface: Describes the structure of a route: method: The HTTP method (e.g., “get”, “post”). path: The URL path (e.g., “/login”). handler: The function that processes the request and sends a response. middlewares: Optional middleware functions to run before the handler.
3. registerRoutes Method: Automatically registers all routes defined in the routes method. Applies middlewares (if any) before the route handler.

```
import { Router, RequestHandler } from "express";

// Define the possible HTTP methods for routes
type RouteMethod = "get" | "post" | "put" | "delete" | "patch";

// Interface to describe the configuration of each route
export interface RouteConfig {
    method: RouteMethod;       // HTTP method (GET, POST, etc.)
    path: string;              // Path for the route
    handler: RequestHandler;   // Request handler for the route (controller met
    middlewares?: RequestHandler[];  // Optional middlewares for this route
}
```

```
// Abstract base class for creating routes
export default abstract class BaseRouter {
    public router: Router;

    // Constructor that initializes the router
    constructor() {
        this.router = Router();  // Create a new Express Router instance
        this.registerRoutes();   // Register routes when the instance is create
    }

    // Abstract method that must be implemented by subclasses to define the rou
    protected abstract routes(): RouteConfig[];

    // Private method that registers all the routes defined in the `routes` met
    private registerRoutes(): void {
        this.routes().forEach(({ method, path, handler, middlewares = [] }) =>
            // Use the appropriate HTTP method to register the route, applying
            this.router[method](path, ...middlewares, handler);
        });
    }
}
```

How to Use BaseRouter?

Extend the BaseRouter class to define routes for specific parts of your application (e.g., authentication, user management).

📁 **src/routes/auth.routes.ts**

```
import AuthController from "@controllers/auth.controller";
import BaseRouter, { RouteConfig } from "./router";
import ValidationMiddleware from "@middlewares/validation.middleware";
import authSchema from "validations/auth.schema";
import AuthMiddleware from "@middlewares/auth.middleware";

class AuthRouter extends BaseRouter {
    protected routes(): RouteConfig[] {
        return [
            {
                // login
                method: "post",
                path: "/login",
                middlewares: [
                    ValidationMiddleware.validateBody(authSchema.login)
                ],
                handler: AuthController.login
            },
            {
```

```
                // register
                method: "post",
                path: "/register",
                middlewares: [
                    ValidationMiddleware.validateBody(authSchema.register)
                ],
                handler: AuthController.register
            },
            {
                // logout
                method: "post",
                path: "/logout",
                middlewares: [
                    // check if user is logged in
                    AuthMiddleware.authenticateUser
                ],
                handler: AuthController.logout
            },

            {
                // refresh token
                method: "post",
                path: "/refresh-token",
                middlewares: [
                    // checks if refresh token is valid
                    AuthMiddleware.refreshTokenValidation
                ],
                handler: AuthController.refreshToken
            },
        ]
    }
}

export default new AuthRouter().router;
```

📁 **src/routes/user.routes.ts**

```
import BaseRouter, { RouteConfig } from "./router";
import AuthMiddleware from "@middlewares/auth.middleware";
import UserController from "@controllers/user.controller";

class UserRoutes extends BaseRouter {
    protected routes(): RouteConfig[] {
        return [
            {
                // get user info
                method: "get",
                path: "/info", // api/user/info
                middlewares: [
```

```
                    AuthMiddleware.authenticateUser
                ],
                handler: UserController.getUser
            },
        ]
    }
}

export default new UserRoutes().router;
```

Why This Structure Matters?

1. **Modularity:** Separates routes into logical groups (e.g., authentication, user management).
2. **Reusability:** Middleware and route logic can be reused across multiple routes.
3. **Scalability:** Adding new routes is straightforward and consistent.

🛠️ Utility: Consistent API Responses with Send

The Send class is a utility that simplifies sending consistent and well-structured API responses. It ensures that all responses follow the same format, making your API easier to use and debug.

📁 **src/utils/response.utils.ts**

```
import { Response } from "express";

class Send {
    static success(res: Response, data: any, message = "success") {
        res.status(200).json({
            ok: true,
            message,
            data
        });
        return;
    }

    static error(res: Response, data: any, message = "error") {
        // A generic 500 Internal Server Error is returned for unforeseen issue
        res.status(500).json({
            ok: false,
            message,
            data,
        });
        return;
```

```
    }

    static notFound(res: Response, data: any, message = "not found") {
        // 404 is for resources that don't exist
        res.status(404).json({
            ok: false,
            message,
            data,
        });
        return;
    }

    static unauthorized(res: Response, data: any, message = "unauthorized") {
        // 401 for unauthorized access (e.g., invalid token)
        res.status(401).json({
            ok: false,
            message,
            data,
        });
        return;
    }

    static validationErrors(res: Response, errors: Record<string, string[]>) {
        // 422 for unprocessable entity (validation issues)
        res.status(422).json({
            ok: false,
            message: "Validation error",
            errors,
        });
        return;
    }

    static forbidden(res: Response, data: any, message = "forbidden") {
        // 403 for forbidden access (when the user does not have the rights to
        res.status(403).json({
            ok: false,
            message,
            data,
        });
        return;
    }

    static badRequest(res: Response, data: any, message = "bad request") {
        // 400 for general bad request errors
        res.status(400).json({
            ok: false,
            message,
            data,
        });
        return;
    }
}
```

```
export default Send;
```

Send class Example:

```
Send.success(res, { id: 1, name: "John" }, "User found");
```

Send class Result

```
{
  "ok": true,
  "message": "User found",
  "data": { "id": 1, "name": "John" }
}
```

🛠️ Validations: Ensuring Data Integrity with Zod

Validations are crucial for ensuring that incoming data meets your application's requirements. Using Zod, a TypeScript-first schema validation library, we can define and enforce rules for request data.

📁 src/validations/auth.schema.ts

This file contains validation schemas for authentication-related requests, such as login and registration.

```
import { z } from "zod";

const passwordSchema = z.string()
    .min(8, "Password must be at least 8 characters long")
    .regex(/[A-Z]/, "Password must include at least one uppercase letter")
    .regex(/[a-z]/, "Password must include at least one lowercase letter")
    .regex(/[0-9]/, "Password must include at least one number")
    .regex(/[@$!%*?&]/, "Password must include at least one special character")

const usernameSchema = z.string()
    .min(6, "Username must be at least 6 characters long")
    .max(20, "Username must not exceed 20 characters")
    .regex(/^[a-zA-Z0-9_-]+$/, "Username can only contain letters, numbers, hyp
    .refine((value) => !/^\d+$/.test(value), {
```

```
            message: "Username cannot be only numbers",
        })
        .refine((value) => !/[@$!%*?&]/.test(value), {
            message: "Username cannot contain special characters like @$!%*?&",
        });

    const login = z.object({
        email: z.string().trim().min(1, "Email is required").email("Invalid email f
        password: z.string().min(1, "Password is required")
    });

    const register = z.object({
        username: usernameSchema,
        email: z.string().email("Invalid email format"),
        password: passwordSchema,
        password_confirmation: z.string().min(1, "Password confirmation is required
    }).refine((data) => data.password === data.password_confirmation, {
        path: ["password_confirmation"],
        message: "Passwords do not match"
    });

    const authSchema = {
        login,
        register
    }

    export default authSchema;
```

Why Use Zod for Validations?

1. **Type Safety:** Zod integrates seamlessly with TypeScript, ensuring type-safe validation.
2. **Readable Schemas:** Validation rules are easy to define and understand.
3. **Custom Error Messages:** Provides clear and user-friendly error messages for invalid data.
4. **Extensibility:** Supports complex validation logic with .refine() and custom rules.

🚀 App Configuration: Setting Up the Express Server

This section covers the core setup of your Express application, including middleware configuration, route registration, and server initialization.

📁 src/app.ts

The App class is the heart of our Express application. It configures middlewares, registers routes, and starts the server.

Key Features:

1. Constructor: Initializes the Express app. Calls initMiddlewares and initRoutes to set up middlewares and routes.

2. initMiddlewares Method: Configures essential middlewares

- express.json(): Parses incoming JSON payloads.
- cookieParser(): Parses cookies attached to requests
- cors(): Enables Cross-Origin Resource Sharing (CORS) for specified frontend URLs (e.g., http://localhost:3000).

3. initRoutes Method: Registers routes for authentication (/api/auth) and user management (/api/user).

4. start Method: Starts the server on the specified host and port (from appConfig). Logs a message when the server is running.

```
import express, { Express } from "express";
import cookieParser from "cookie-parser";
import cors from "cors";
import authRoutes from "@routes/auth.routes";
import appConfig from "@config/app.config";
import userRoutes from "@routes/user.routes";

class App {
    private app: Express;

    constructor() {
        this.app = express()

        this.initMiddlewares();
        this.initRoutes();
    }

    private initMiddlewares() {
        this.app.use(express.json());
        this.app.use(cookieParser());
        this.app.use(cors({
            origin: [
                'http://localhost:3000', // your frontend url
                'https://mywebsite.com' // your production url optional
            ],
            methods: ["GET", "POST", "DELETE"],
            credentials: true
```

```
        }))
    }

    private initRoutes() {
        // /api/auth/*
        this.app.use("/api/auth", authRoutes);
        // /api/user/*
        this.app.use("/api/user", userRoutes);
    }

    public start() {
        const { port, host } = appConfig;

        this.app.listen(port, host, () => {
            console.log(`server is running on http://${host}:${port}`);

        })
    }
}

export default App;
```

📁 **src/db.ts**

This file configures the Prisma client for database interactions.

Key Features:

1. Singleton Pattern: Ensures only one instance of the Prisma client is created, improving performance.
2. Environment-Specific Behavior: Prevents re-initialization of the Prisma client in development environments.

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = global as unknown as { prisma: PrismaClient };

export const prisma =
  globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

📁 **src/index.ts**

This is the entry point of your application. It initializes the App class and starts the server.

```
import "tsconfig-paths/register";
import "dotenv/config"

import App from "app";

const app = new App;

app.start();
```

🧪 How to Test the Application

Once you've set up the project and implemented the code, you can test the authentication and authorization flow to ensure everything works as expected. Here's a step-by-step guide to testing the application:

1. Start the Development Server

Run the following command to start the development server:

```
npm run dev
```

This command starts your Express.js server using **Nodemon**, a tool that **automatically restarts** the server whenever you make changes to the code inside the src/ folder.

2. Test the API Endpoints

You can use tools like **Postman**, **Insomnia**, or **cURL** to test the API endpoints. Below are the key endpoints and how to test them:

a. User Registration

- **Endpoint:** `POST /api/auth/register`
- **Request Body:**

```
{
  "username": "john28",
```

```
  "email": "john@example.com",
  "password": "Password@123",
  "password_confirmation": "Password@123"
}
```

b. User Login

- **Endpoint:** `POST /api/auth/login`
- **Request Body:**

```
{
  "email": "john@example.com",
  "password": "Password@123"
}
```

- **Cookies:**
- `accessToken`: A short-lived JWT token (e.g., 15 minutes).
- `refreshToken`: A long-lived JWT token (e.g., 24 hours).

c. Fetch User Information

- **Endpoint:** `GET /api/user/info`
- **Headers:** Ensure the `accessToken` cookie is sent with the request.
- **Response**

```
{
  "ok": true,
  "message": "success",
  "data": {
    "user": {
      "id": 1,
      "username": "john_doe",
      "email": "john@example.com",
      "createdAt": "2023-10-01T12:00:00.000Z",
      "updatedAt": "2023-10-01T12:00:00.000Z"
    }
  }
}
```

d. Refresh Access Token

- **Endpoint:** `POST /api/auth/refresh-token`
- **Headers:** Ensure the `refreshToken` cookie is sent with the request.
- **Response:**

```
{
  "ok": true,
  "message": "Access token refreshed successfully"
}
```

- **Cookies:** A new `accessToken` cookie will be set.

e. User Logout

- **Endpoint:** `POST /api/auth/logout`
- **Headers:** Ensure the `accessToken` and `refreshToken` cookies are sent with the request.
- **Response**

```
{
  "ok": true,
  "message": "Logged out successfully."
}
```

3. Verify Token Revocation

After logging out, try accessing the `GET /api/user/info` endpoint again. You should receive a `401 Unauthorized` error, indicating that the access token is no longer valid.

4. Test Database Integration

Check the database (e.g., MySQL, PostgreSQL) to ensure that:

- User records are created during registration.
- Refresh tokens are stored and cleared correctly during login and logout.

Happy coding! 🚀

Expressjs + Jwt Authentication + Jwt + Typescript + Prisma +

Follow

Written by gigi shalamberidze

9 followers · 64 following

React Developer with 4 years of frontend and 2 years of backend experience, specializing in Next.js, Express.js, and building high-performance web apps.